

CSCE 5612 Embedded Hardware/Software Design

Spring 2026

Project Milestone Two: System Architecture, Firmware Design, and Rule-Based Pipeline

Project Title: Laser Beam Alarm Security System

with Blynk Cloud Monitoring & Gmail Alerts

Udayasai Ramineni – 11802872

Asfaaq Ahamed Kapatrala- 11786442

Submission Date: April 3, 2026

University of North Texas

1. Use-Case Scenarios and System Objectives

1.1 Real-World Scenario

In order to prevent any illegal entry into restricted zones, a system for securing entrance points needs to be put in place. As part of this project, we have constructed a laser beam trip wire alarm which constantly checks if the laser beam aimed at a light dependent resistor (LDR) is broken or not by an intruder in the secure area.

This alarm is meant to be used in indoor fixed applications like the entrance doors of homes, server rooms, storerooms, or laboratories among other secured entrances. The alarm works constantly using USB power from a wall adapter and sounds a local as well as remote alarm upon breaking of the beam.

1.2 Problem Statement

The current PIR motion sensors respond to changes in temperature and thus are not effective in detecting slowly moving intruders. The simple contact sensors can detect only whether the door or window is open or closed but do not sense an open path between the two points. This research aims to address these problems through the use of a highly focused laser light with analog LDR sensor detection.

1.3 System Role — No Machine Learning

This design does not involve any machine learning algorithm. All the logic behind detecting whether the beam is broken or not is rule-based and embedded into the firmware running on the Raspberry Pi Pico W. This decision was made on purpose because:

- The detection task is a clear binary one – either there is an intrusion, or there isn't.
- It is enough to use a static threshold and apply a filter based on consecutive readings (3 in 300 ms).
- Rule-based algorithms work deterministically without requiring any training dataset or consuming significant resources.
- ML can only make the system unnecessarily complicated for solving this simple task.

1.4 Expected Outcomes

Embedded System Outcomes:

- Can monitor LDR ADC value continuously every 100 ms
- Local alarm triggered within one polling cycle (~100 ms) of detecting beam interruption (Buzzer + Flashing Red LED)
- Cloud Dashboard updates in remote Blynk application within ~500 ms of triggering local alarm
- Gmail notification sent within ~3 seconds of triggering alarm (SMTP connection established already)
- Self-resetting system upon restoration of beam

Detection Logic Outcomes:

Beam-break detection category: NORMAL mode versus ALARM mode

- False alarm rejection via consecutive readings filter (3+ readings exceeding threshold necessary)
- Threshold adjusted dynamically during startup using actual laser and LDR reading

1.5 Requirements vs. Specifications

Requirement (General Goal)	Specification / Constraint (Measurable)
Detect beam interruption quickly	Detection latency ≤ 300 ms (3×100 ms polling interval)
Provide local audible and visual alert	Buzzer ≥ 85 dB; LEDs visible in normal indoor lighting
Send remote cloud alert	Blynk dashboard updates within 500 ms; Gmail within 3 seconds
Minimize false alarms	≥ 3 consecutive ADC readings above threshold required before alarm
Operate continuously without user intervention	24/7 powered via USB 5V/2A wall adapter; no battery required
Fit within RP2040 resource constraints	Firmware < 100 KB Flash; RAM usage < 50 KB at runtime
Maintain Wi-Fi connection without interruption	Persistent TCP socket to Blynk; reconnect on disconnect
Total system cost $< \$50$	BOM sourced from Adafruit + Amazon; PCB from JLCPCB

2. System Architecture and Firmware Design

2.1 System Block Diagram

The block diagram presented below depicts the full hardware and software design of the Laser Beam Alarm System, which includes sensors, actuators, the Pico W processor, communication links, and cloud service systems.

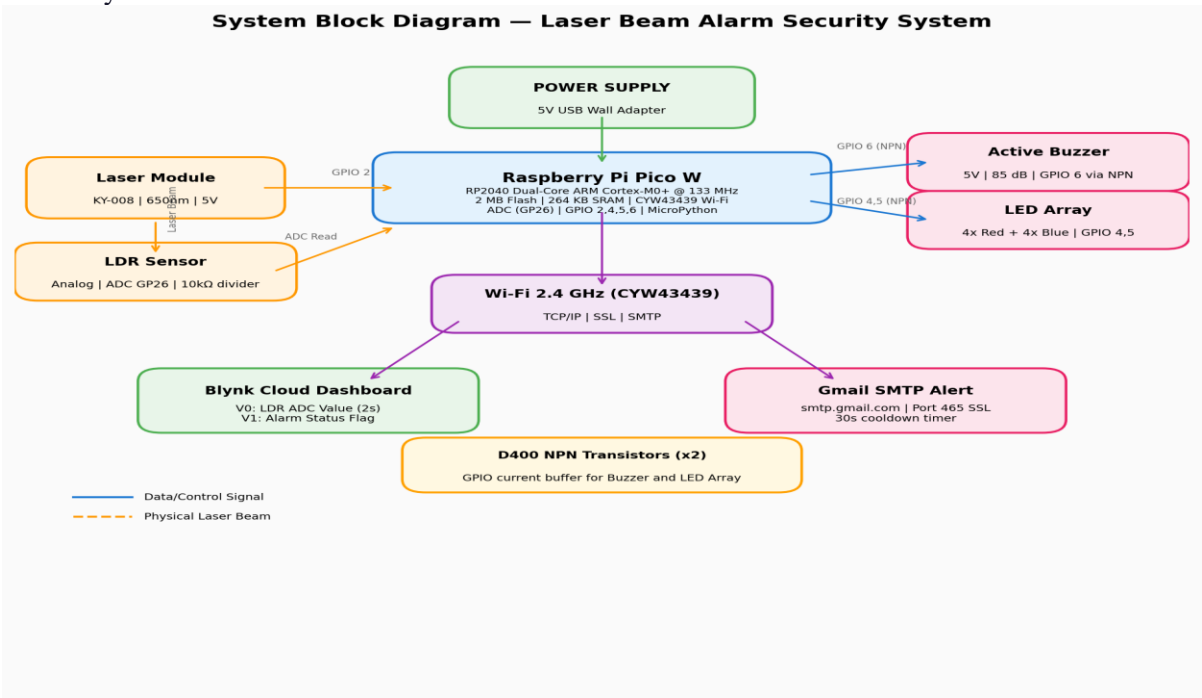


Figure 1: System Block Diagram — Laser Beam Alarm Security System

2.2 Data Flow Diagram

The following data flow diagram highlights the information flow from the LDR to ADC conversion, threshold comparison, local alarm triggering, and cloud notification.

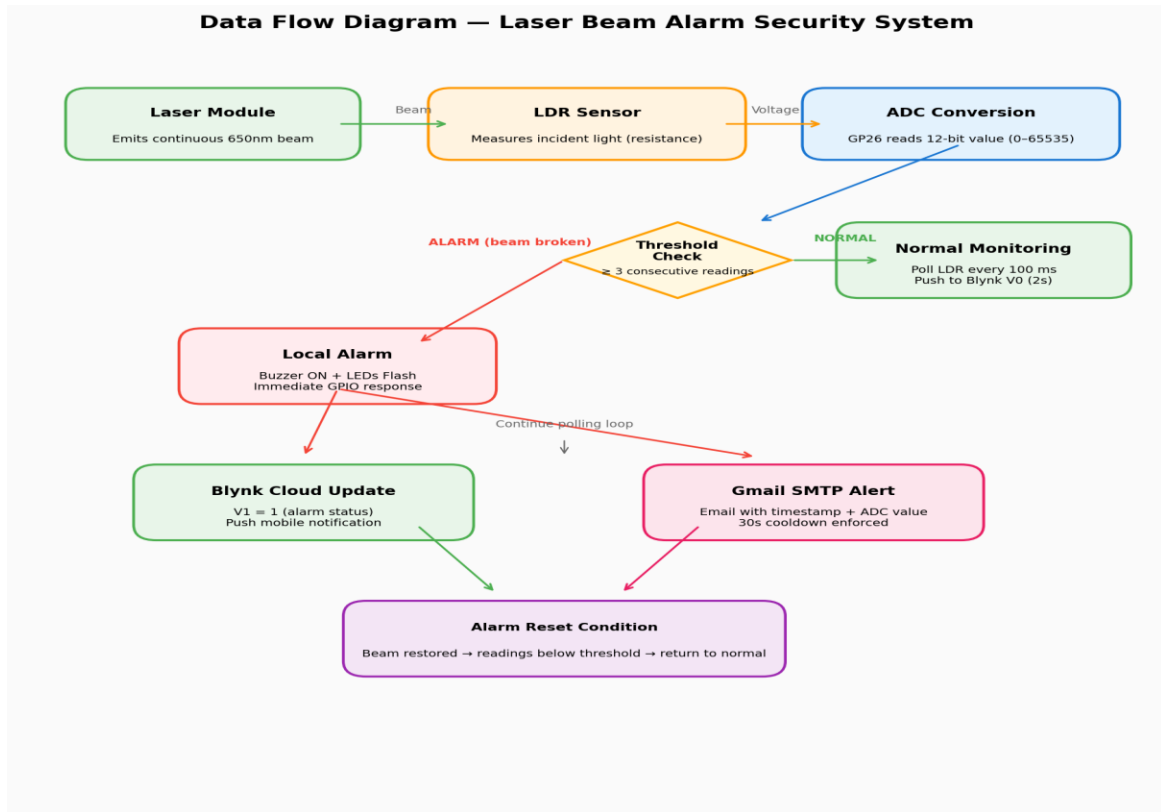


Figure 2: Data Flow Diagram — From Sensor to Output

2.3 Firmware Flowchart

The flowchart for the firmware contains all the main program logic starting from the process of initializing the system and ending at the resetting process.

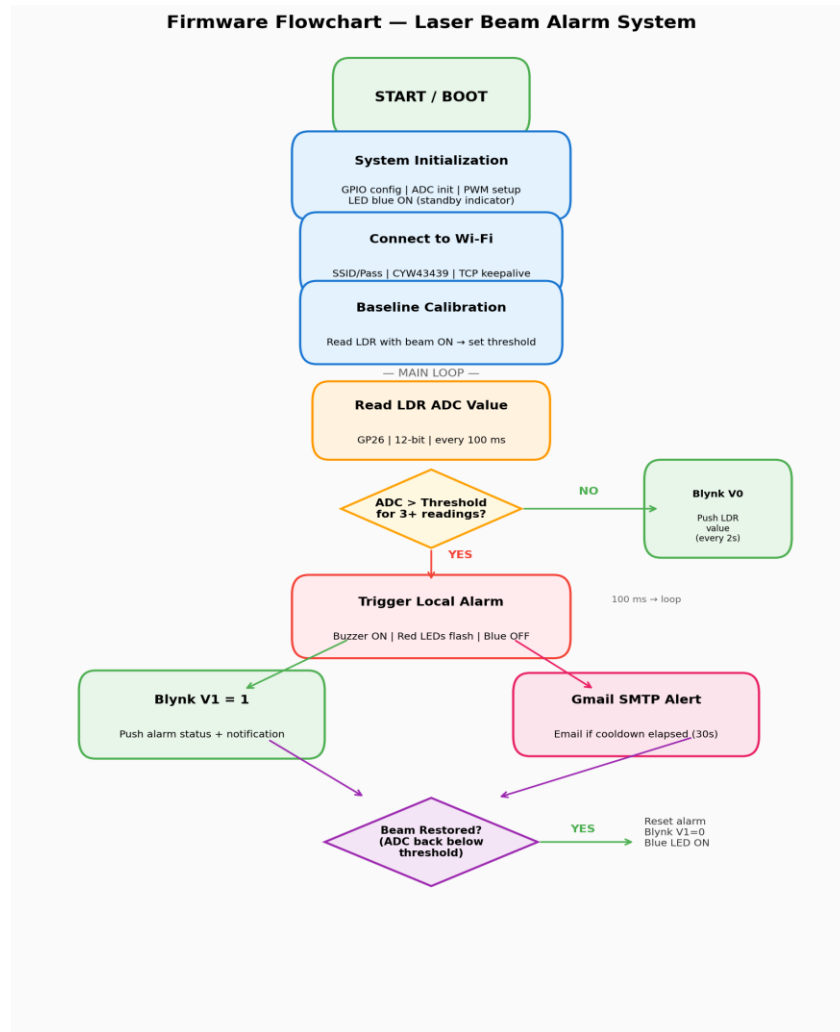


Figure 3: Firmware Flowchart — Main Program Logic

2.4 Software Module Diagram

Module diagram for the software is depicted below, with details of the modules, their functionalities, and interaction between them. The structure consists of Hardware Abstraction Layer, which forms the base, followed by modules, and the final layer is the firmware main loop.

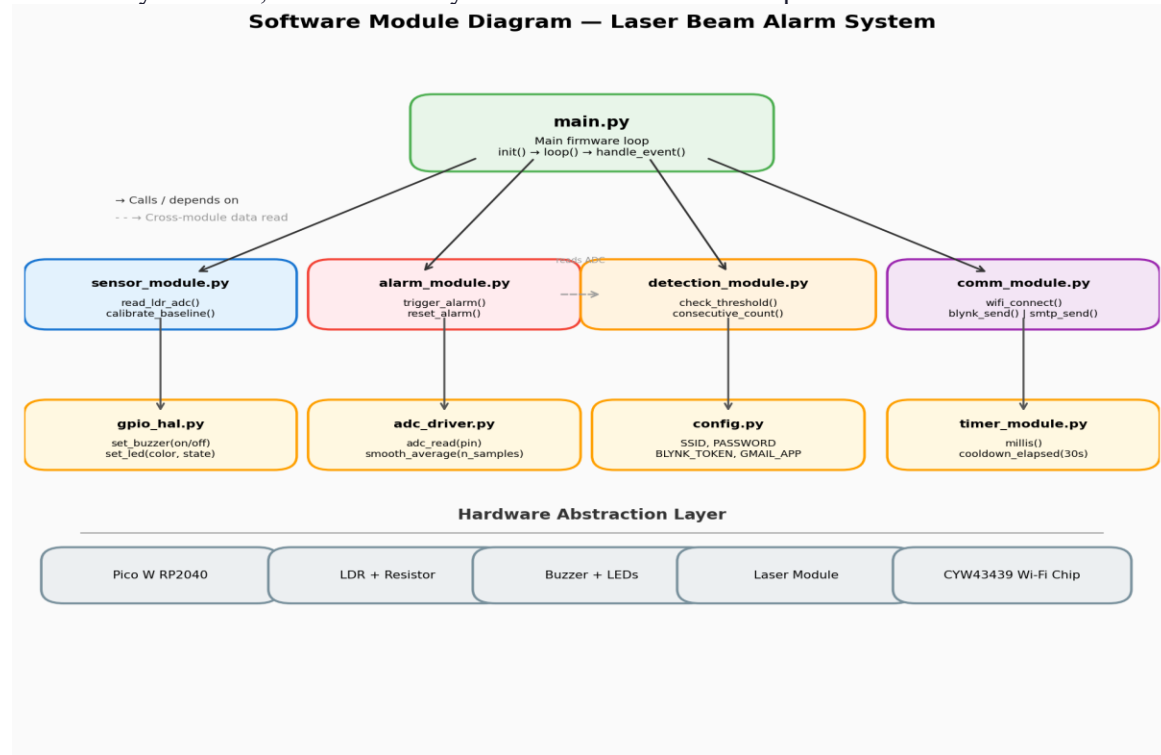


Figure 4: Software Module Diagram and Call Graph

2.5 System Control Behavior

Sequential Behavior:

At the start-up, the code follows a set pattern during initialization: pin settings for GPIO, ADC setup, PWM settings for LED control, Wi-Fi setup, Blynk socket connection, and LDR base calibration. This procedure must be followed sequentially before entering the main loop.

Conditional Logic:

The primary loop contains threshold comparison logic for the ADC value at all times. The number of consecutive counts greater than the threshold is stored internally by the counter. The ALARM case will be triggered if there are three counts greater than the threshold. On lower than threshold counts, the counter will be reset to zero by the NORMAL case.

Iterative Behavior:

The loop responsible for the sampling process runs infinitely, and reads the values from the LDR through its ADC every 100 milliseconds with a timer delay. Another separate counter monitors the passage of time to indicate when the values from the LDR should be sent to Blynk.

Interrupt-Driven / Timer-Based Events:

The timing is handled using the MicroPython `utime.sleep_ms(100)` method for the polling delay and `utime.ticks_ms() / utime.ticks_diff()` for the Blynk reporting delay and Gmail delay period timer. The hardware interrupt mechanism is not involved; instead, only polling-based operations are implemented, which is justified by the required latency of 100 ms.

2.6 Firmware Integration of Detection Logic

The detection algorithm based on rules works as follows:

Reasoning Trigger: The detection occurs for every ADC sample (sampling period = 100 ms). There is no need for a separate trigger mechanism.

Data Window: An increment counter of 3 threshold crossings (300 ms window) serves as an alarm event. No buffering of data is necessary.

Action upon detecting an event: ALARM triggers GPIO output (buzzer and red LEDs turn ON), GPIO turns Blynk V1=1, and Gmail SMTP email is sent out. NORMAL maintains the value of the LDR on Blynk V0 and blue LED ON every 2 seconds.

2.7 Implementation Considerations

Timing constraints: LDR polling at 100 ms is sufficient enough to detect intrusions with the help of human speed. Wi-Fi and SMTP processes are non-blocking as compared to the local alarm activation which occurs instantly using GPIO. **Memory constraints:** MicroPython firmware requires roughly 256 to 512 KB memory out of the total 2 MB of Flash memory. Memory consumption at runtime will be around 20 to 40 KB which fits well in the available 264 KB of SRAM in RP2040.

Power constraints: Worst-case consumption of power is about 736 mW (refer to Milestone 1 analysis). 5V/2A adapter on USB offers power of 10 W. Wi-Fi radio keeps running in the mode of persistent connection for low reconnection latency. **Fault tolerance:** Disconnection of the Wi-Fi radio can be found using the Blynk library. The firmware tries to reconnect before proceeding ahead. SMTP failures are handled with the help of try/except statement. Even in case of failure in cloud communication, the system will activate the local alarm.

Modularity: Each of the subsystems mentioned above is separately written in Python modules, making the testable and exchangeable components.

3. Data Collection Plan

3.1 Data to Be Collected

As there is no ML involved, the system requires no training dataset. The operational data collected by the system during its implementation include the following:

- LDR ADC raw reading (12-bit; range from 0 to 65535) sampled at 10 Hz (every 100 milliseconds)
- Timestamps for the occurrence and resumption of an alarm event
- LDR ADC raw reading at the time of the alarm event
- Calibration value upon startup of the system

3.2 Hardware Modules Used for Data Collection

Component	Data Type	Interface	Sampling Rate
LDR Sensor (GP26)	12-bit ADC integer (0–65535)	Analog ADC	10 Hz (100 ms)
Raspberry Pi Pico W	Timestamp (ms since boot)	Internal RTC / utime	Continuous
Blynk Cloud	Virtual pin log (LDR value + alarm status)	Wi-Fi TCP	V0: 0.5 Hz; V1: on-event
Gmail SMTP	Alarm event email record	Wi-Fi SSL/SMTP	On-event (30s cooldown)

3.3 Data Format and Storage

Locally – The ADC value is stored as an integer in a continuously updated 3 element list (`consecutive_readings[]`). There is no requirement for persistent storage since the RP2040 doesn't write any data to the flash memory in normal working conditions to increase the lifetime.

On the Cloud (Blynk) – All the updates of virtual pins by Blynk automatically include the UTC timestamp along with logging. V0 (Light Dependent Resistor Value) updates occur every 2 seconds, whereas V1 (Alarm Status) logs all the transitions. Blynk maintains the historical records that can be accessed through the application and dashboard.

Email notifications (Gmail) – Every alarm creates a simple text email with the subject “Laser Alarm Triggered” which contains the time stamp and ADC value along with the system status when it occurred.

3.4 Collection Protocol

Initial calibration: Ten readings from the ADC are taken per boot cycle while maintaining the laser beam to determine the base level. The threshold for the alarm is determined at 50% above the base level (for example, if base level = 3000, then threshold = 4500). This calibration changes based on installation site conditions.

Data collection: Data is collected continuously upon operation. There are no time limits for sessions; the system operates forever.

Test cases: Beam breaks were simulated manually under various speeds, including hand movement, obstacle blocking, and total blocking, throughout the development stage to test the reliability of detection and overall latency.

Tested environmental settings: Normal lighting indoors, low lighting conditions, and outdoor lighting, where the threshold can be verified in different ambient lighting conditions.

Special test cases: Transient beam breaking for less than 200 milliseconds will not trigger an alarm (filtered by three reading requirements), and sustained beam breaking beyond 300 milliseconds will trigger an alarm.

3.5 Data Labeling

Since there is no use of ML models, no need for formal labels arises. But to test the validity of the system, alarms are marked manually as:

- TRUE POSITIVE: Beam interruption due to presence of any person or object
- FALSE POSITIVE: Triggering of alarms due to changes in environmental light or vibrations
- TRUE NEGATIVE: Normal condition maintained by the system in the absence of beam interruptions
- FALSE NEGATIVE: In case the beam is interrupted, but no alarm is raised

3.6 Data Quality and Privacy

Data quality: ADC readings are checked against plausible range (0–65535). Readings outside valid range are discarded and flagged in firmware. The 3-reading filter inherently rejects single-sample noise spikes.

Synchronization: Single-sensor system; no multi-sensor synchronization is required. Timestamps use `utime.ticks_ms()` referenced to system boot time.

Privacy/Security: Gmail credentials (app-specific password) are stored as constants in firmware. Communication occurs over SSL (port 465). Blynk uses token-based authentication over TLS. No personally identifiable information is transmitted.

Data balance: Not applicable. No classifier training is performed.

4. Data Processing, Detection Logic, and Deployment Plan

4.1 Preprocessing Steps

All preprocessing occurs in firmware on the Pico W microcontroller:

Data Quality: ADC values are compared to a reasonable range (0-65535). Invalid data points are ignored. The three-point filter effectively removes random spikes from the sample.

Synchronization: Single sensor setup; no multi-sensor synchronization is needed. Time stamps utilize `utime.ticks_ms()` relative to the system boot time.

Privacy and Security: Email password and app password are stored as constants in the firmware. Communication utilizes SSL (port 465). Blynk utilizes token-based access over TLS. No personal information is shared.

Data Balance: Not relevant. Classifier training does not apply

4.2 Feature Extraction

There are no extractions of features. The algorithm directly works on the analog-to-digital converter output values by performing the threshold test as explained above. The one computed value is that of deviation from the baseline level.

4.3 Detection Algorithm

The detection algorithm is a simple threshold classifier with hysteresis implemented as a consecutive-reading filter:

Step	Operation	Implementation
1	Read LDR ADC	<code>adc.read_u16()</code> on GP26, averaged over 3 rapid samples
2	Compare to threshold	<code>if adc_val > baseline * 1.5: consecutive_count += 1 else: consecutive_count = 0</code>
3	Alarm decision	<code>if consecutive_count >= 3: trigger alarm</code>
4	Alarm reset	<code>if alarm_active and adc_val < threshold: alarm_active = False; reset outputs</code>

Rationale for this technique includes:

Deterministic and real-time: $O(1)$ processing per cycle; latency only up to the polling period

Negligible memory consumption except for a single integer counter

Does not require any training or calibration apart from a single baseline reading at initialization

Adaptable: configurable constant parameters are `threshold_multiplier` and `consecutive_count_threshold` in `config.py`

4.4 Deployment Plan

This implementation has a completely client-based architecture. It contains neither any offline component nor any cloud-side inference algorithm:

client-based (Pico W): Detection algorithms, preprocessing steps, GPIO handling, Wi-Fi setup, Blynk connectivity, and SMTP email generation are all implemented and executed locally using MicroPython on the RP2040 chip.

Server-based (Blynk): The role of the Blynk cloud is that of being a passive data repository and dashboard only – all processing happens client-side. It just stores and displays data received from Pico W.

Server-based (Gmail): Gmail servers receive and store alert emails. Apart from delivery no processing happens server-side.

Deployment options and processes:

- MicroPython version 1.22+ flashed to Pico W using Thonny IDE over USB
- Blynk MicroPython library (blynklib_mp) installed on Pico W file system
- usocket and ssl MicroPython standard libraries for SMTP over SSL
- TensorFlow Lite, Edge Impulse, or scikit-learn are irrelevant to this system and thus not used

4.5 Optimizations

Polling period: The use of 100 ms polling period allows for appropriate latency while making the main loop more efficient. Any further reduction would increase CPU usage without any significant benefits.

Limiting Blynk sending: Blynk V0 receives LDR reading once every two seconds (once every 20 polling periods) to reduce the number of API calls as well as to reduce the Wi-Fi radio utilization.

Permanent Wi-Fi socket: Permanent socket to Blynk API ensures fast handling of alarms by not requiring reconnecting. Wi-Fi radio remains active even in normal modes.

SMTP cooldown: 30-second cooldown between SMTP requests prevents too many SMTP requests in case of blockages and reduces Wi-Fi radio loading.

5. Evaluation Plan

5.1 Detection Logic Evaluation

Since there is no ML model, evaluation focuses on the rule-based detection performance using the standard binary classification metrics applied to manually labeled test scenarios:

Metric	Definition	Target
True Positive Rate (Recall)	Beam-break events correctly triggering alarm	$\geq 99\%$
False Positive Rate	Non-intrusion events incorrectly triggering alarm	$< 1\%$
Detection Latency	Time from beam break to local alarm actuation	≤ 300 ms
Reset Latency	Time from beam restoration to alarm reset	≤ 200 ms
Cloud Notification Latency	Time from alarm trigger to Blynk update	≤ 500 ms
Email Alert Latency	Time from alarm trigger to Gmail delivery	≤ 5 seconds

5.2 Confusion Matrix Approach

Testing scenarios are performed manually with a predetermined set of 50 test cases comprising 25 positive instances (hand movements, obstructions of the beam, etc., at different velocities) and 25 negative instances (ambient light variations, vibrations, temporary light flickers). The test results are categorized as follows:

- True Positive (TP): Beam broken, signal triggered;
- False Negative (FN): Beam broken, signal NOT triggered (false negatives);
- True Negative (TN): Beam NOT broken, signal NOT triggered;
- False Positive (FP): Beam NOT broken, signal triggered (false positives).

The calculation formula for Precision ($TP/(TP+FP)$), Recall ($TP/(TP+FN)$), and F1 Score ($2*(Precision*Recall)/(Precision+Recall)$) is used to evaluate the system performance.

5.3 Embedded System Evaluation

Latency: Measured using a logic analyzer hooked up to GPIO 6 (buzzer pin), and timing using a stopwatch starting from when the physical beam break occurs. Goal: Latency ≤ 300 ms.

Memory usage: Memory usage measured using MicroPython's `gc.mem_alloc()` and `gc.mem_free()` functions at runtime. Flash memory usage is measured using Thonny's file size reporting feature. Goal: Less than 50 KB of RAM used.

Real-time testing: The system is tested over 24 hours of continuous runtime to ensure polling loop reliability, reliable connection to Blynk, and lack of memory leaks (by calling `gc.collect()`).

Resistance to noise: The system is tested with flickering of overhead lighting, adding reflective surfaces close to the beam path, and vibrating the PCB slightly. The system should not raise any alarms under these conditions.

Fake alarms per hour: Alarms raised in one hour in a regular office environment with no interruptions to the beam. Goal: No false alarms.

Power consumption: Power consumption measured using a USB power meter (e.g., UM25C) while the system operates normally and under an alarm condition. Goal: Consumption < 800 mW.

5.4 Testing Strategy

Test on hardware: All testing for functions will be done on the assembled hardware. This is the major testing approach, as the system relies on hardware components.

Test simulation / offline test: The ADC threshold detector is unit tested in a Python script on the computer using simulation of the ADC values. Tests include normal input, spike input, breach input, and recovery tests.

Offline testing via replay of recorded data: Recorded data from the hardware test can be replayed into the detector logic module.

Sensor and Module Summary Table

Module/Sensor	Interface	Sampling Rate	Data Type	Purpose
LDR Sensor (GP26)	ADC	10 Hz (100 ms)	12-bit integer (0–65535)	Beam-break detection input
Laser Module (GPIO2)	Digital GPIO Output	Continuous (always ON)	ON/OFF	Beam source
Active Buzzer (GPIO6)	Digital GPIO Output via NPN	On alarm event	ON/OFF	Local audible alert
Red LEDs (GPIO5)	GPIO PWM Output via NPN	On alarm event	ON/FLASH/OFF	Visual alarm indicator
Blue LEDs (GPIO4)	GPIO PWM Output	Continuous standby	ON/OFF	Standby status indicator
Blynk V0	Wi-Fi TCP	0.5 Hz (2s)	Float (LDR value)	Remote LDR monitoring
Blynk V1	Wi-Fi TCP	On event	Integer (0 or 1)	Remote alarm status
Gmail SMTP	Wi-Fi SSL	On event (30s cooldown)	Email text	Email alert notification

Memory and Computation Estimate

Resource	Estimate	Available (RP2040)	Margin
MicroPython firmware + runtime	~512 KB Flash	2,048 KB	75% free
Application code (all .py files)	~20–30 KB Flash	Remaining Flash	Ample
Runtime RAM (heap, stack, MicroPython VM)	~40–60 KB	264 KB SRAM	~200 KB free
ADC sample buffer (3 integers)	< 100 bytes	Part of heap	Negligible
Consecutive count + state variables	< 50 bytes	Part of heap	Negligible
Blynk TCP socket buffer	~2–4 KB	Part of heap	Within margin
Detection computation per cycle	O(1), < 0.1 ms	133 MHz CPU	Negligible
Polling cycle budget (100 ms)	~0.5 ms used	100 ms available	99.5% idle